Academic Year: 2026-27 Sap Id: 60003240292

Shri Vile Parle Kelavani Mandal'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous) Affiliated to the University of Mumbai

Department of Information Technology

COURSE CODE: DJS23ICPC501 **DATE: 13-08-2026**

COURSE NAME: Computer Networks **CLASS: T. Y. B.Tech**

BATCH: I3-1

Experiment No. 5

CO/LO: Analyse various concerns at different layers in reference models using suitable techniques.

Aim: Write a program to implement CRC code for error detection and correction

Theory:

Cyclic Redundancy Check or CRC is a method of detecting accidental changes/errors in the communication channel. CRC uses **Generator Polynomial** which is available on both sender and receiver side.

An example generator polynomial is of the form like x3 + x + 1. This generator polynomial represents key 1011. Another example is x2 + 1 that represents key 101. There are two primary variables in CRC:

- ***n:*** *Number of bits in data to be sent from sender side*
- ***k:*** *Number of bits in the key obtained from generator polynomial.*

Encoded Data Generation from Generator Polynomial (Sender Side)

- The binary data is first augmented by adding k-1 zeros in the end of the data.
- Then, modulo - 2 binary division is used to divide binary data by the key and remainder of division is stored.
- At last the the remainder is appended at the end of the data to form the encoded data which is later sent.

Checking Error in Transmission (Receiver Side)

- After receiving the data, to check if the data is error free, perform the modulo-2 division again.
- If the remainder is 0, then there are not errors, otherwise, the data is faulty and contain transmission errors.

Modulo 2 Division

The process of modulo-2 binary division is the same as the familiar division process we use for decimal numbers. Just that instead of subtraction, we use XOR here.

- In each step, a copy of the divisor (or data) is XORed with the k bits of the dividend (or key).

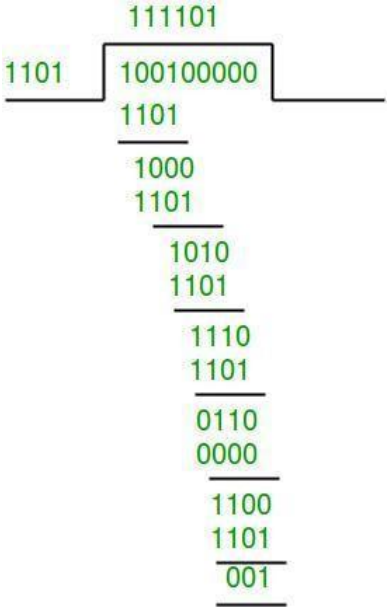
- The result of the XOR operation (remainder) is (k-1) bits, which is used for the next step after 1 extra bit is pulled down to make it k bits long.
- When there are no bits left to pull down, we have a result. The (k-1) bit remainder which is appended at the sender side.

Examples:

Case 1: No error in transmission

Data = 100100, Generator Polynomial (Key) = x3 + x2 + 1 (1101)

Sender Side

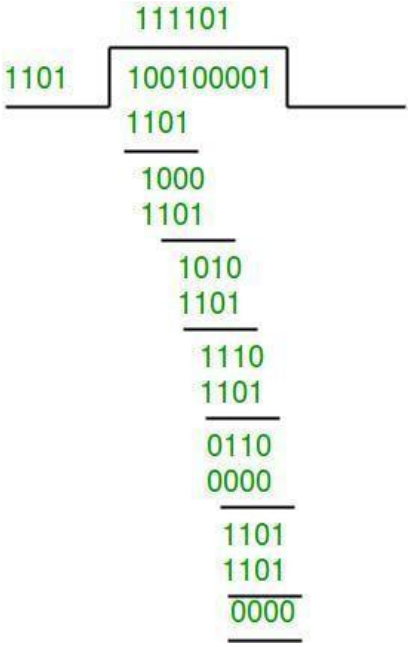


Generating Remainder

The remainder is 001. Thus the data sent is **100100001.**

Receiver Side

Code word received at the receiver side **100100001**

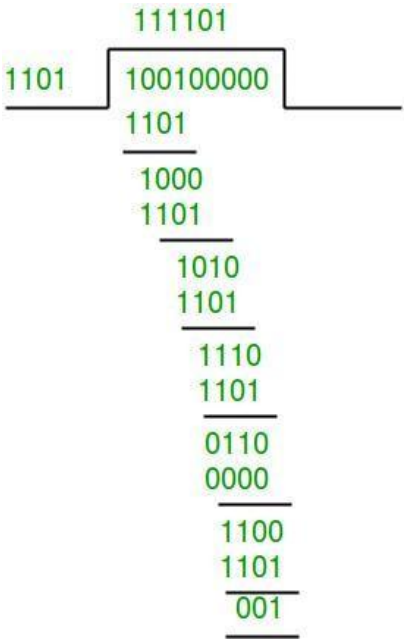


Checking the Remainder

The remainder is 0, hence the data received has no errors.

Case 2: Error in Transmission

Data = 100100, Generator Polynomial (Key) = x3 + x2 + 1 (1101)

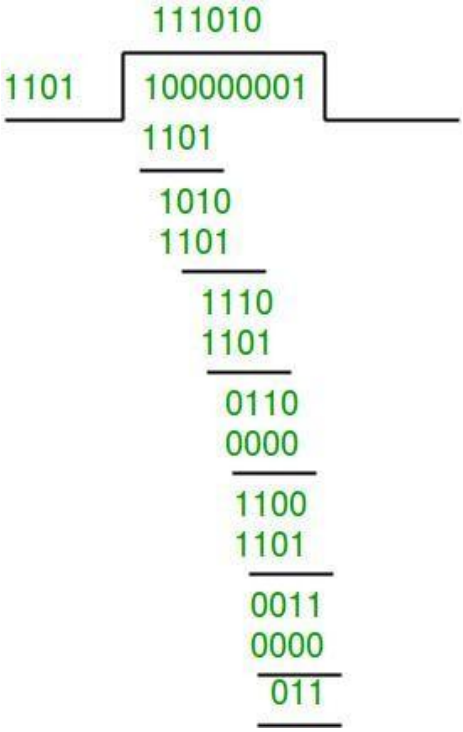


Sender Side

*The remainder is 001. Thus the data sent is **100100001.***

Receiver Side

*Let there be an error and code word received at the receiver side **100000001.***



As the remainder is not 0, hence there is some error detected in the receiver side.

EXERCISE

Your Program should do the following.

1. Read a text file and generate CRC codes for a predefined code length.
2. Verify the received data code for any error.
3. Read and image file and convert it into its binary equivalent.
4. Apply CRC code on the Image for a predefined size.
5. Verify the received data code for any error.

Code:

```
def crc(data, generator):
    data = list(data + '0' * (len(generator) - 1))

    for i in range(len(data) - len(generator) + 1):
        if data[i] == '1':
            for j in range(len(generator)):
                data[i + j] = str(int(data[i + j]) ^ int(generator[j]))

    return ''.join(data[-(len(generator) - 1):])

message = input("Enter message: ")
generator = input("Enter generator: ")

remainder = crc(message, generator)
sent_data = message + remainder

print("Remainder:", remainder)
print("Sent data:", sent_data)

received_data = input("Enter received data: ")
check = crc(received_data, generator)

print("Check remainder:", check)

if int(check) == 0:
    print("Valid")
else:
    print("Invalid")
```

Output:



Observation:
The CRC program successfully performs error detection by taking a binary message and generator polynomial as input, calculating the CRC remainder using modulo-2 division, and appending the remainder to the original message to form the transmitted data. At the receiver side, the received data is divided again by the same generator. A zero remainder indicates that the received data is valid and free from detected transmission errors, while a non-zero remainder indicates that an error has occurred during transmission.

Conclusion:
The experiment successfully demonstrates the implementation of the Cyclic Redundancy Check (CRC) technique for detecting errors in computer networks. CRC provides an effective and reliable method for identifying accidental changes in transmitted data by using polynomial division and XOR operations. The program verifies the received data by comparing the resulting remainder with zero, thereby demonstrating how CRC is used for error detection at the receiver side.

Observation Questions:

1. Consider the message M= 1010001101. Let the divisor polynomial be x5+x4+x2+1. Find the cyclic redundancy code for this message.
2. Obtain a 4 bit CRC code for the data bit sequence 10011011100 using the polynomial x4+ x2+1
   You are given a 16-bit data word: 1101101010101101 and a CRC polynomial:
   x^5 + x^2 + 1. Compute the CRC for this data using CRC-CCITT (16-bit CRC).

Reference:

[1] www.geeksforgeeks.org